

Modelagem, Implementação e Análise Comparativa de Estruturas em Árvore Especializadas

Aluno: Thales Guerreiro Lima Gomes de Castro

Disciplina: Algoritmos e Estruturas de Dados II

Professor: Michel Pires

Resumo

Este trabalho apresenta um estudo comparativo de cinco estruturas de dados em árvore especializadas: Trie, árvore Patricia, árvore Splay, Treap e KD-Tree. O objetivo é compreender os princípios de organização, as propriedades estruturais, as operações fundamentais, as complexidades assintóticas e os contextos de aplicação de cada estrutura. Foram desenvolvidas implementações próprias das cinco estruturas, contemplando operações de inserção, busca e remoção, além das características específicas de cada abordagem. A avaliação experimental foi realizada para tamanhos de entrada de 100, 1.000, 10.000 e 100.000 elementos, considerando conjuntos aleatórios, ordenados e invertidos. Foram medidos os tempos de inserção, busca de elementos existentes, busca de elementos inexistentes e remoção. Os resultados evidenciaram diferenças significativas entre as estruturas e entre os padrões de entrada, destacando particularmente a sensibilidade da Splay à sequência de acessos e a variação observada na Treap para entradas ordenadas e invertidas. A Trie apresentou comportamento relativamente estável nas maiores entradas avaliadas, enquanto Patricia, Splay, Treap e KD-Tree apresentaram maior variação entre determinadas operações e cenários. Os resultados reforçam que a escolha de uma estrutura de dados deve considerar não somente sua complexidade assintótica, mas também a natureza das chaves, a distribuição dos dados e o padrão das operações realizadas.

Palavras-chave: estruturas de dados; árvores; Trie; Patricia; Splay; Treap; KD-Tree; análise de complexidade; benchmarking.

1 Introdução e Fundamentação Teórica

Estruturas de dados hierárquicas são fundamentais para a organização e recuperação eficiente de informações. Entre as estruturas mais conhecidas encontram-se as árvores binárias de busca (Binary Search Trees – BST), nas quais cada nó possui uma chave e a organização dos elementos obedece à propriedade de que as chaves menores são posicionadas à esquerda e as maiores à direita. Árvores AVL ampliam esse conceito ao introduzir mecanismos explícitos de balanceamento, buscando limitar a altura da árvore e, conseqüentemente, manter operações de busca, inserção e remoção em tempo logarítmico.

Apesar da importância das BST e AVL, nem todos os problemas de organização e consulta de dados podem ser adequadamente representados por uma única estratégia. Algumas aplicações trabalham predominantemente com strings e prefixos; outras apresentam padrões de acesso não uniformes; outras necessitam de balanceamento probabilístico; e outras ainda lidam com pontos em espaços multidimensionais. Essas diferenças motivam o estudo de estruturas especializadas.

O presente trabalho concentra-se em cinco estruturas com características distintas: Trie, Patricia, Splay, Treap e KD-Tree. A Trie foi introduzida como uma estrutura voltada à representação e recuperação de informações organizadas segundo os caracteres que compõem suas chaves [2]. A Patricia, proposta por Morrison, busca uma representação eficiente para indexação e recuperação de informações, reduzindo estruturas de ramificação desnecessariamente longa [3]. A Splay é uma árvore binária de busca autoajustável que reorganiza a estrutura durante os acessos [5]. A Treap combina as propriedades de uma árvore binária de busca com as de um heap, utilizando prioridades para produzir um comportamento probabilístico de balanceamento [4]. Finalmente, a KD-Tree estende a ideia de busca binária para espaços multidimensionais, sendo apropriada para consultas espaciais [1].

O objetivo geral é, portanto, implementar, analisar e comparar essas cinco estruturas, relacionando suas propriedades formais ao comportamento observado experimentalmente. O trabalho não trata as estruturas como cinco implementações isoladas, mas como diferentes estratégias para problemas de organização e recuperação de dados.

1.1 Motivação

A principal motivação para estudar estruturas especializadas é compreender que o desempenho de uma estrutura de dados depende não somente do número de elementos armazenados, mas também da natureza das chaves, do padrão de consultas, da dimensionalidade dos dados e das propriedades estruturais mantidas durante as operações.

Uma Trie, por exemplo, não organiza strings por comparação lexicográfica entre chaves completas, mas explora os prefixos compartilhados. Uma Patricia busca representar essa mesma ideia de forma mais compacta. A Splay não mantém um fator de balanceamento explícito, mas modifica dinamicamente a estrutura de acordo com os acessos. A Treap utiliza uma prioridade adicional para combinar duas propriedades de ordenação. Já a KD-Tree utiliza diferentes coordenadas para dividir recursivamente um espaço multidimensional.

Dessa maneira, a escolha de uma estrutura deve considerar o problema que se pretende resolver, e não apenas a complexidade assintótica de uma operação isolada.

1.2 Trie

Uma Trie é uma árvore especializada na representação de conjuntos de strings. Cada caminho a partir da raiz representa uma sequência de caracteres, sendo que prefixos compartilhados por diferentes palavras ocupam a mesma parte da estrutura [2].

Considere, por exemplo, as palavras 'car', 'card' e 'care'. As três compartilham o prefixo 'car', portanto os caracteres correspondentes a esse prefixo podem ser armazenados em um caminho comum. A existência de uma palavra é indicada por uma informação associada ao nó correspondente ao último caractere da chave.

Na implementação desenvolvida neste trabalho, cada nó possui um conjunto de ponteiros para os possíveis caracteres do alfabeto utilizado e uma indicação de término de palavra. A implementação utilizada considera inicialmente o alfabeto de letras minúsculas, de modo que cada caractere é convertido em um índice do vetor de filhos.

As operações principais são inserção, busca e remoção. A estrutura também permite verificar a existência de um prefixo e pode ser utilizada para consultas nas quais todas as palavras iniciadas por determinado prefixo precisam ser recuperadas.

Se uma chave possui comprimento m , as operações de busca e inserção percorrem, no máximo, m posições correspondentes aos caracteres da chave. Para um alfabeto de tamanho constante, isso resulta em custo $O(m)$. A remoção possui a mesma ordem em relação ao comprimento da chave, pois os nós são examinados durante o percurso de volta da recursão.

A principal vantagem da Trie é a exploração explícita dos prefixos. Sua principal limitação está no consumo de memória, especialmente em implementações que mantêm um vetor fixo de ponteiros para cada nó.

1.3 Árvore Patricia

A implementação utilizada neste trabalho representa as chaves como inteiros e utiliza os bits das chaves para determinar os pontos de divergência. A função `firstDifferentBit` identifica o primeiro bit no qual duas chaves diferem, e esse bit é utilizado para determinar a posição da nova ramificação na árvore Patricia. A motivação central é evitar caminhos formados por sucessivos nós que não apresentam uma decisão relevante. Morrison apresentou a estrutura como uma técnica de armazenamento, indexação e recuperação de informações com preocupação especial com o espaço utilizado pelo índice [3].

Enquanto uma Trie tradicional pode possuir um nó para cada caractere de uma chave, uma representação Patricia pode compactar sequências que não provocam bifurcações. Assim, a estrutura tende a utilizar menos nós para determinados conjuntos de chaves.

A operação de inserção exige identificar a região de maior correspondência entre a nova chave e os prefixos já presentes. Quando duas chaves compartilham somente parte de seus caracteres, pode ser necessário dividir um segmento existente e criar uma nova ramificação. Essa divisão constitui um dos mecanismos mais importantes da estrutura.

A busca percorre os segmentos relevantes da chave, verificando as correspondências necessárias. A remoção, por sua vez, pode exigir a reorganização ou recomposição de segmentos quando uma ramificação deixa de ser necessária.

A complexidade da Patricia depende da representação adotada. Para chaves de comprimento limitado por m , as operações fundamentais podem ser analisadas em função do número de posições ou caracteres efetivamente examinados. Em uma implementação baseada em strings compactadas, a busca possui custo associado ao comprimento da chave, enquanto a compactação reduz a quantidade de nós necessários em comparação com uma Trie não compactada em conjuntos com muitos caminhos de grau um.

A Patricia apresenta, portanto, um compromisso entre a organização por prefixos da Trie e a necessidade de reduzir o espaço estrutural utilizado.

1.4 Árvore Splay

A Splay Tree é uma árvore binária de busca autoajustável proposta por Sleator e Tarjan [5]. Diferentemente de uma AVL, a Splay não mantém explicitamente um fator de balanceamento ou uma altura limitada por uma regra estrutural. Sua principal característica é a operação de *splaying*, que reorganiza a

árvore após um acesso.

Quando um elemento é acessado, a estrutura realiza uma sequência de rotações destinada a mover o nó acessado para a raiz. Dependendo da posição relativa entre o nó, seu pai e seu avô, são utilizadas operações conhecidas como *zig*, *zig-zig* e *zig-zag*.

A estratégia possui uma propriedade importante: embora uma operação individual possa apresentar custo $O(n)$ no pior caso, uma sequência de operações possui custo amortizado $O(\log n)$ por operação [5]. Dessa forma, a eficiência não é obtida pela manutenção de uma condição de balanceamento após cada operação, mas pela reorganização adaptativa da árvore.

Na implementação desenvolvida, as operações de rotação são mecanismos internos utilizados pelo procedimento de *splay*. As operações de busca, inserção e remoção são realizadas respeitando a propriedade de árvore binária de busca e utilizando a reorganização característica da estrutura.

Essa característica torna a Splay particularmente interessante para conjuntos de dados em que determinados elementos são acessados repetidamente. Após sucessivos acessos, esses elementos tendem a permanecer próximos da raiz, reduzindo o custo de acessos posteriores.

1.5 Treap

A Treap combina duas propriedades. Em relação às chaves, mantém a propriedade de uma árvore binária de busca; em relação às prioridades, mantém a propriedade de um heap. A estrutura é associada ao conceito de *Randomized Search Tree*, estudado por Seidel e Aragon [4].

Cada nó possui, portanto, pelo menos duas informações de ordenação: a chave utilizada pela propriedade de BST e uma prioridade utilizada pela propriedade de heap. A prioridade pode ser gerada de forma pseudoaleatória.

Durante uma inserção, a chave é posicionada respeitando a propriedade de busca. Em seguida, rotações podem ser utilizadas para restaurar a propriedade de heap em relação à prioridade. A remoção também pode utilizar rotações para conduzir o nó até uma posição na qual possa ser removido preservando as propriedades restantes da estrutura.

O comportamento esperado de busca, inserção e remoção é $O(\log n)$ quando as prioridades são aleatórias e independentes, enquanto o pior caso pode chegar a $O(n)$ caso a estrutura se torne degenerada. A quantidade de memória é $O(n)$.

Uma vantagem importante da Treap é a relativa simplicidade de sua regra de balanceamento. Em vez de calcular fatores de balanceamento como na AVL, a estrutura utiliza prioridades e rotações para manter simultaneamente duas propriedades de ordenação.

1.6 KD-Tree

A KD-Tree, ou árvore k -dimensional, é uma generalização da ideia de busca binária para dados organizados em um espaço multidimensional. Bentley apresentou a estrutura como uma forma de realizar buscas associativas sobre registros multidimensionais [1].

Em uma KD-Tree bidimensional, por exemplo, um nível pode utilizar a coordenada x para dividir os pontos, enquanto o nível seguinte utiliza a coordenada y . O processo continua alternando as dimensões de acordo com a profundidade.

Dessa maneira, cada nó representa não somente um ponto, mas também uma divisão do espaço. Em duas dimensões, essa divisão pode ser visualizada como uma sequência de regiões que particionam o plano.

Além da busca por um ponto exato, uma das principais vantagens da KD-Tree é permitir consultas espaciais. Entre elas estão buscas por região e busca pelo vizinho mais próximo.

A complexidade depende fortemente da dimensão e da forma da árvore. Para dados adequadamente distribuídos e uma árvore com altura aproximadamente logarítmica, a inserção pode apresentar comportamento médio próximo de $O(\log n)$. Entretanto, uma estrutura degenerada pode atingir $O(n)$. Consultas espaciais também dependem da dimensionalidade, da distribuição dos pontos e da quantidade de regiões que precisam ser exploradas.

A KD-Tree, portanto, representa uma mudança importante em relação às demais estruturas deste trabalho: sua principal motivação não é organizar strings ou chaves unidimensionais, mas permitir particionamento e consulta eficiente de espaços multidimensionais.

2 Projeto e Implementação das Estruturas

As cinco estruturas foram implementadas em C++, utilizando alocação dinâmica de memória e organização modular. O objetivo da implementação foi manter cada estrutura suficientemente independente

para permitir testes individuais e, posteriormente, a execução de um benchmark padronizado.

A implementação priorizou as operações fundamentais e as operações específicas de cada estrutura. O código completo, os programas de teste e os arquivos necessários para reprodução dos experimentos serão disponibilizados no repositório do projeto.

2.1 Trie

A representação da Trie utiliza um nó contendo um vetor de ponteiros para os filhos e uma informação booleana indicando se aquele nó representa o final de uma palavra.

A inserção percorre a chave caractere por caractere. Caso o filho correspondente ao caractere ainda não exista, um novo nó é criado. Ao final da chave, o nó correspondente é marcado como término de palavra.

A busca executa o mesmo percurso, porém sem criar novos nós. Caso algum caminho necessário não exista, a chave não está presente. Ao alcançar o último caractere, o resultado depende da marcação de término de palavra.

A remoção é realizada recursivamente. Ao alcançar o final da chave, a marcação de término é removida. Em seguida, durante o retorno da recursão, nós que não possuem filhos e que também não representam o final de outra palavra podem ser desalocados.

Além das operações fundamentais, a implementação contempla a verificação de existência de prefixo. Essa operação percorre os caracteres do prefixo e verifica se o caminho correspondente existe, sem exigir que o último nó seja necessariamente o fim de uma palavra.

2.2 Patricia

A Patricia utiliza uma representação compactada dos caminhos associados às chaves. O principal mecanismo de implementação é a identificação das partes comuns entre as chaves e a divisão dos segmentos quando uma nova chave diverge de uma chave já armazenada.

A inserção pode produzir três situações principais: a nova chave pode seguir integralmente um caminho existente; uma chave existente pode ser prefixo da nova chave; ou as duas chaves podem compartilhar apenas parte de seus prefixos, exigindo uma divisão.

A busca compara os segmentos relevantes da chave até determinar se a chave está presente. A remoção deve preservar as propriedades de compactação, evitando manter ramificações que não sejam mais necessárias.

Essa implementação evidencia a principal diferença em relação à Trie: enquanto a Trie utiliza explicitamente os caracteres como níveis da árvore, a Patricia procura armazenar a informação de maneira mais compacta.

2.3 Splay

A implementação da Splay utiliza a representação tradicional de uma árvore binária de busca. Cada nó possui uma chave e referências para os filhos esquerdo e direito.

A operação de *splay* é realizada por rotações. Quando o nó acessado possui o pai como raiz, ocorre um caso *zig*. Quando o nó e seu pai estão no mesmo lado em relação ao avô, ocorre um caso *zig-zig*. Quando estão em lados opostos, ocorre um caso *zig-zag*.

A busca percorre a árvore seguindo a propriedade de BST e, após encontrar o elemento, realiza o *splay* para conduzi-lo à raiz. A inserção e a remoção também utilizam a reorganização da árvore para preservar o comportamento autoajustável.

As rotações são implementadas como operações auxiliares da estrutura, não como operações de alto nível destinadas ao usuário.

2.4 Treap

A Treap utiliza nós contendo uma chave, uma prioridade e os ponteiros para os filhos esquerdo e direito.

Durante a inserção, a chave é inicialmente posicionada segundo a propriedade de BST. A prioridade é então utilizada para determinar se rotações são necessárias. O processo continua até que a propriedade de heap seja restaurada.

A remoção utiliza a combinação das propriedades de BST e heap para conduzir o nó que será removido a uma posição adequada. As rotações desempenham papel central nesse procedimento.

A geração das prioridades é realizada por um mecanismo pseudoaleatório. Para garantir maior consis-

tência nos experimentos, a estratégia de geração das prioridades deve ser mantida igual entre diferentes execuções do benchmark, quando o objetivo for comparar estruturas sobre o mesmo conjunto de chaves.

2.5 KD-Tree

A KD-Tree foi implementada utilizando pontos em espaço multidimensional. A dimensão utilizada durante uma inserção depende da profundidade do nó.

Para uma estrutura bidimensional, por exemplo:

$$\text{dimensao} = \text{profundidade} \bmod 2. \quad (1)$$

Assim, em um nível a comparação pode ser realizada utilizando x e, no nível seguinte, utilizando y .

Além das operações de inserção e busca, a implementação contempla consultas específicas relacionadas à natureza espacial da estrutura, incluindo busca por região e busca do ponto mais próximo, conforme definido no projeto.

A consulta por região explora somente regiões que podem conter pontos pertencentes ao espaço consultado. A busca pelo vizinho mais próximo mantém a melhor distância encontrada e utiliza os limites das regiões para decidir se subárvores adicionais precisam ser exploradas.

3 Demonstração e Rastreamento Visual

A visualização das estruturas foi planejada para evidenciar não somente seu estado final, mas também as transformações produzidas pelas operações.

O enunciado do trabalho determina que sejam apresentados estados inicial, intermediário e posterior a operações relevantes, dando especial atenção à reorganização da Splay e da Treap, à compactação da Patricia, ao compartilhamento de prefixos da Trie e ao particionamento espacial da KD-Tree.

3.1 Trie

Para a Trie, um exemplo adequado utiliza as chaves:

car, card, care, cat, dog

O diagrama deve evidenciar que as palavras iniciadas por “ca” compartilham parte da estrutura.

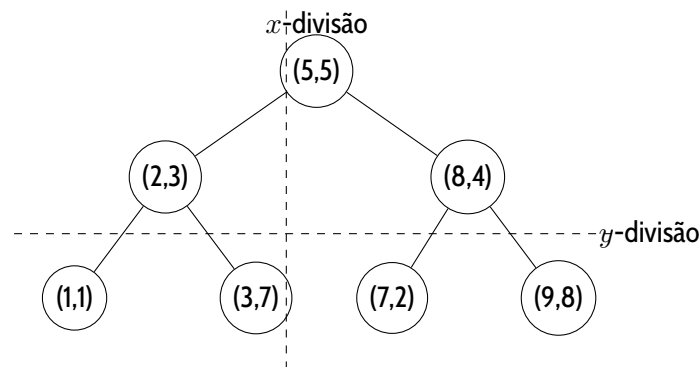


Figura 1: Exemplo bidimensional de KD-Tree e de seu particionamento alternado por coordenadas.

Essa representação é importante porque demonstra visualmente a diferença entre a KD-Tree e as estruturas voltadas à organização de chaves unidimensionais.

4 Análise de Complexidade e Comparação Teórica

A análise de complexidade permite comparar as estruturas considerando não somente o custo assintótico das operações, mas também o tipo de dado e as propriedades necessárias para que determinado comportamento seja obtido.

Para Trie e Patricia, é importante considerar o comprimento da chave, representado por m . Para as árvores de busca, considera-se principalmente o número de elementos n . Para a KD-Tree, a dimensão k também influencia o custo das consultas espaciais.

4.1 Tabela comparativa

A Tabela 1 resume os principais custos assintóticos.

Tabela 1: Comparação teórica das principais operações.

Estrutura	Operação	Melhor	Médio/Esperado	Pior	Espaço
Trie	Busca	$O(1)$	$O(m)$	$O(m)$	$O(S\Sigma)$
Trie	Inserção	$O(1)$	$O(m)$	$O(m)$	$O(S\Sigma)$
Trie	Remoção	$O(1)$	$O(m)$	$O(m)$	$O(S\Sigma)$
Trie	Prefixo	$O(1)$	$O(m + r)$	$O(m + r)$	$O(S\Sigma)$
Patricia	Busca	$O(1)$	$O(m)$	$O(m)$	$O(n)$
Patricia	Inserção	$O(1)$	$O(m)$	$O(m)$	$O(n)$
Patricia	Remoção	$O(1)$	$O(m)$	$O(m)$	$O(n)$
Splay	Busca	$O(1)$	$O(\log n)$ amortizado	$O(n)$	$O(n)$
Splay	Inserção	$O(1)$	$O(\log n)$ amortizado	$O(n)$	$O(n)$
Splay	Remoção	$O(1)$	$O(\log n)$ amortizado	$O(n)$	$O(n)$
Treap	Busca	$O(1)$	$O(\log n)$ esperado	$O(n)$	$O(n)$
Treap	Inserção	$O(1)$	$O(\log n)$ esperado	$O(n)$	$O(n)$
Treap	Remoção	$O(1)$	$O(\log n)$ esperado	$O(n)$	$O(n)$
KD-Tree	Busca	$O(\log n)$	$\approx O(n^{1-1/k})$	$O(n)$	$O(n)$
KD-Tree	Inserção	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$
KD-Tree	Busca espacial	Variável	$\approx O(n^{1-1/k} + r)$	$O(n)$	$O(n)$

Na tabela, m representa o comprimento da chave, Σ representa o tamanho do alfabeto, S representa o número de nós da Trie, n representa o número de elementos e r representa a quantidade de resultados reportados. Para a KD-Tree, k representa a dimensionalidade do espaço.

Os valores apresentados devem ser interpretados de acordo com as hipóteses de cada estrutura. Em particular, a complexidade esperada da Treap depende da distribuição das prioridades, enquanto a Splay possui garantia amortizada e não garantia de tempo logarítmico para cada operação individual [5, 4].

4.2 Comparação com BST e AVL

A BST convencional apresenta custo de busca, inserção e remoção proporcional à altura da árvore. Em uma árvore degenerada, a altura pode ser $O(n)$, fazendo com que as operações também apresentem esse custo.

A AVL introduz uma restrição explícita sobre o balanceamento e mantém altura $O(\log n)$, garantindo que busca, inserção e remoção permaneçam logarítmicas.

A Splay apresenta uma abordagem diferente. Não existe uma restrição estrutural equivalente à AVL; em vez disso, os acessos provocam reorganização. Assim, uma operação individual pode ser linear, mas uma sequência de operações possui custo amortizado $O(\log n)$ [5].

A Treap também não mantém o balanceamento por meio de alturas ou fatores de balanceamento. As prioridades introduzem uma componente probabilística. Quando as prioridades são aleatórias, a altura esperada é logarítmica, embora uma configuração desfavorável possa produzir uma árvore degenerada.

Trie e Patricia não são diretamente substitutas de BST e AVL em todos os problemas. Elas são especialmente apropriadas quando as chaves possuem estrutura textual ou quando a relação entre prefixos é relevante. Nessas situações, o comprimento da chave pode ser mais importante que o número total de elementos.

A KD-Tree, por sua vez, trata de uma classe distinta de problemas. Sua organização é determinada pelas coordenadas dos pontos e não por uma ordenação total simples. Por isso, sua comparação com BST e AVL deve ser feita principalmente em relação ao tipo de consulta que cada estrutura consegue realizar.

5 Metodologia Experimental e Resultados

A avaliação experimental foi realizada com o objetivo de observar o comportamento prático das cinco estruturas implementadas: Trie, Patricia, Splay, Treap e KD-Tree. O benchmark foi executado utilizando o mesmo conjunto de tamanhos e os mesmos cenários de entrada para todas as estruturas, respeitando as particularidades de representação de cada uma.

Foram avaliadas quatro operações: inserção, busca de elementos existentes, busca de elementos inexistentes e remoção. O tempo de execução de cada operação foi medido separadamente, utilizando milissegundos como unidade de medida.

5.1 Conjuntos de dados

Foram utilizados quatro tamanhos de entrada:

$$n \in \{100, 1000, 10000, 100000\}.$$

Para cada tamanho foram considerados três cenários de distribuição das chaves:

- **aleatório**: as chaves de 1 até n foram embaralhadas utilizando uma semente fixa;
- **ordenado**: as chaves foram inseridas na ordem crescente, de 1 até n ;
- **invertido**: as chaves foram inseridas na ordem decrescente, de n até 1.

A utilização de uma semente fixa no embaralhamento permite reproduzir o mesmo conjunto aleatório entre diferentes execuções do experimento.

Para a Trie, as chaves inteiras utilizadas pelo benchmark foram convertidas para uma representação composta por letras minúsculas, de acordo com o alfabeto suportado pela implementação. Para a KD-Tree, cada chave foi associada a um ponto bidimensional (x, y) , sendo a primeira coordenada relacionada à chave e a segunda obtida a partir de uma permutação determinística.

As buscas de elementos existentes utilizaram todas as chaves inseridas na estrutura. Para as buscas de elementos inexistentes foram utilizadas chaves no intervalo de $n + 1$ até $2n$, ou seus equivalentes na representação utilizada pela estrutura.

Tabela 2: Configuração dos experimentos realizados.

Parâmetro	Valores
Tamanho da entrada (n)	100, 1.000, 10.000, 100.000
Cenários	aleatório, ordenado, invertido
Operações	inserção, busca existente, busca inexistente, remoção
Estruturas	Trie, Patricia, Splay, Treap, KD-Tree

No total, foram avaliadas $4 \times 3 \times 5 = 60$ combinações de estrutura, tamanho e cenário, sendo realizadas quatro medições de tempo para cada combinação.

5.2 Procedimento de medição

Para cada combinação de estrutura, tamanho e cenário, o benchmark construiu inicialmente a estrutura com o conjunto correspondente de dados. Em seguida, cada operação foi medida separadamente.

Na medição de inserção, o tempo registrado corresponde ao período necessário para inserir todos os elementos do conjunto. Na busca de elementos existentes, a estrutura foi construída novamente e todas as chaves foram consultadas. O mesmo procedimento foi utilizado para a busca de elementos inexistentes.

Para a remoção, uma nova estrutura foi construída antes da medição e todos os elementos foram removidos.

O tempo de preparação dos dados, geração das chaves e impressão dos resultados não foi incluído na medição das operações. Os resultados foram armazenados em arquivo CSV contendo as colunas correspondentes à estrutura, tamanho, cenário e tempos das quatro operações.

Os valores apresentados a seguir correspondem à execução realizada no ambiente de desenvolvimento utilizado para o trabalho. Como os tempos foram medidos em uma única execução, valores muito pequenos devem ser interpretados com cautela, pois podem sofrer influência de fatores externos, como escalonamento do sistema operacional, estado dos caches e variações ocasionais de execução.

5.3 Resultados de inserção

A Tabela 3 apresenta os tempos de inserção registrados para os quatro tamanhos de entrada. Para manter a apresentação compacta, são mostrados os valores correspondentes ao cenário aleatório.

Tabela 3: Tempo de inserção no cenário aleatório, em milissegundos.

Estrutura	$n = 100$	$n = 1.000$	$n = 10.000$	$n = 100.000$
Trie	0,0536	0,4571	2,4562	23,4639
Patricia	0,0682	0,3496	3,2246	58,0239
Splay	0,0382	0,2065	3,6292	82,6586
Treap	0,0579	0,2743	6,4973	147,2500
KD-Tree	0,0211	0,2185	4,0524	115,2340

No cenário aleatório, observa-se crescimento do tempo de inserção conforme o número de elementos aumenta. A Trie apresentou crescimento relativamente regular, passando de 0,0536 ms para 23,4639 ms entre $n = 100$ e $n = 100.000$.

A Patricia também apresentou crescimento, chegando a 58,0239 ms para 100.000 elementos. Splay, Treap e KD-Tree apresentaram tempos maiores nessa escala, respectivamente 82,6586 ms, 147,2500 ms e 115,2340 ms.

Os resultados mostram que, para essa implementação e para o cenário aleatório, o crescimento observado não é determinado somente pela complexidade assintótica. Custos de alocação dinâmica, operações de ponteiros, rotações e características da representação de cada estrutura também influenciam os tempos medidos.

No caso da Treap, o tempo de inserção aumentou de 6,4973 ms para 147,2500 ms entre $n = 10.000$ e $n = 100.000$. Esse resultado deve ser interpretado em conjunto com os demais cenários, pois uma única execução não permite separar completamente os efeitos da implementação das variações ocasionais do ambiente de execução.

5.4 Resultados de busca

A busca foi dividida em duas situações: elementos que pertencem à estrutura e elementos que não pertencem à estrutura. Essa separação permite verificar se a ausência da chave produz comportamento significativamente diferente.

Tabela 4: Tempo de busca de elementos existentes no cenário aleatório, em milissegundos.

Estrutura	$n = 100$	$n = 1.000$	$n = 10.000$	$n = 100.000$
Trie	0,0012	0,0269	1,1243	5,7059
Patricia	0,0146	0,0958	1,4667	45,9322
Splay	0,0156	0,1176	3,4286	78,5408
Treap	0,0062	0,0398	1,4364	26,3607
KD-Tree	0,0063	0,0524	0,8536	43,9425

Para o cenário aleatório, a Trie apresentou os menores tempos de busca de elementos existentes nas maiores escalas avaliadas, com 5,7059 ms para $n = 100.000$. A Treap apresentou 26,3607 ms, enquanto Patricia, KD-Tree e Splay apresentaram, respectivamente, 45,9322, 43,9425 e 78,5408 ms.

Na Splay, o resultado deve ser interpretado considerando sua característica autoajustável. A estrutura modifica sua forma durante os acessos, de modo que o custo de uma operação individual pode variar significativamente de acordo com o estado atual da árvore e a sequência de acessos.

Tabela 5: Tempo de busca de elementos inexistentes no cenário aleatório, em milissegundos.

Estrutura	$n = 100$	$n = 1.000$	$n = 10.000$	$n = 100.000$
Trie	0,0017	0,0210	0,4794	2,3153
Patricia	0,0125	0,1069	1,0282	53,2343
Splay	0,0025	0,0127	0,1027	1,2511
Treap	0,0012	0,0030	0,0446	2,1345
KD-Tree	0,0019	0,0133	0,0827	1,6260

Um resultado particularmente interessante aparece na busca de elementos inexistentes. Para $n = 100.000$, a Patricia apresentou 53,2343 ms, enquanto Trie, Splay, Treap e KD-Tree apresentaram, respectivamente, 2,3153, 1,2511, 2,1345 e 1,6260 ms.

Essa diferença demonstra que o custo prático da busca depende não somente da existência ou ausência

da chave, mas também da forma como cada implementação percorre sua estrutura e da representação utilizada.

5.5 Resultados de remoção

A Tabela 6 apresenta os resultados da remoção para o cenário aleatório.

Tabela 6: Tempo de remoção no cenário aleatório, em milissegundos.

Estrutura	$n = 100$	$n = 1.000$	$n = 10.000$	$n = 100.000$
Trie	0,0090	0,2246	1,0112	22,8651
Patricia	0,0341	0,2394	2,8763	97,3098
Splay	0,0290	0,2465	5,4343	60,5319
Treap	0,0268	0,2142	7,2755	502,4250
KD-Tree	0,0329	0,2212	2,6714	47,2835

A remoção apresentou diferenças ainda mais acentuadas entre as estruturas. Para $n = 100.000$ no cenário aleatório, a Trie apresentou 22, 8651 ms, enquanto Patricia apresentou 97, 3098 ms e Splay apresentou 60, 5319 ms.

A Treap apresentou 502, 4250 ms nesse cenário, valor consideravelmente superior aos demais. Esse comportamento evidencia que, na implementação avaliada, o custo prático das operações de remoção pode ser fortemente influenciado pela forma da árvore e pelas operações de reorganização necessárias.

5.6 Influência do cenário de entrada

A distribuição das chaves produziu diferenças significativas em algumas estruturas. O efeito mais evidente ocorreu na Treap para $n = 100.000$. No cenário aleatório, a inserção levou 147, 2500 ms, enquanto no cenário ordenado chegou a 1110, 1600 ms e no cenário invertido a 1137, 4200 ms.

As buscas da Treap também apresentaram diferenças relevantes nessa escala. A busca de elementos existentes levou 26, 3607 ms no cenário aleatório, 509, 3810 ms no ordenado e 493, 9290 ms no

invertido. Para elementos inexistentes, os tempos foram 2, 1345 ms, 1295, 4800 ms e 0, 5330 ms, respectivamente.

Esses resultados mostram que a distribuição dos dados pode ter efeito substancial no comportamento observado, mesmo quando a análise assintótica indica um comportamento esperado logarítmico. No caso de estruturas probabilísticas, a forma concreta produzida pelas prioridades e a sequência de operações podem afetar significativamente os custos medidos.

A Splay apresentou comportamento diferente. Para $n = 100.000$, sua inserção foi de 82, 6586 ms no cenário aleatório, mas caiu para 7, 9674 ms no cenário ordenado e para 5, 6819 ms no cenário invertido. As buscas existentes apresentaram, respectivamente, 78, 5408, 7, 1638 e 9, 3461 ms.

Esse comportamento é compatível com a natureza adaptativa da Splay: sua estrutura é alterada pelas próprias operações realizadas e, portanto, a ordem dos acessos interfere diretamente no estado da árvore.

A Trie apresentou variação menor entre os cenários. Para $n = 100.000$, a inserção foi de 23, 4639 ms no cenário aleatório, 13, 4124 ms no ordenado e 23, 9258 ms no invertido. Isso é coerente com o fato de que sua organização depende principalmente dos caracteres das chaves, e não de uma propriedade de ordenação numérica global.

5.7 Resultados no cenário ordenado

O cenário ordenado é particularmente útil para observar estruturas de busca que podem ser influenciadas pela ordem de inserção.

Para $n = 100.000$, a Splay apresentou 7, 9674 ms de inserção, 7, 1638 ms de busca existente, 1, 1706 ms de busca inexistente e 10, 1115 ms de remoção.

A Treap apresentou comportamento bastante diferente: 1110, 1600 ms de inserção, 509, 3810 ms de busca existente e 1295, 4800 ms de busca inexistente. A remoção, entretanto, apresentou apenas 7, 4042 ms.

A KD-Tree apresentou 129, 1300 ms de inserção, 73, 7303 ms de busca existente, 8, 0668 ms de busca inexistente e 82, 9158 ms de remoção.

A Trie apresentou os menores tempos entre as estruturas avaliadas nesse cenário em diversas operações: 13, 4124 ms na inserção, 2, 8322 ms na busca existente, 1, 6225 ms na busca inexistente e 28, 4579

ms na remoção.

5.8 Resultados no cenário invertido

No cenário invertido, para $n = 100.000$, a Trie apresentou 23, 9258 ms de inserção, 7, 1901 ms de busca existente, 2, 7679 ms de busca inexistente e 22, 9416 ms de remoção.

A Splay apresentou 5, 6819 ms de inserção, 9, 3461 ms de busca existente, 2, 6520 ms de busca inexistente e 17, 6400 ms de remoção.

A Treap apresentou 1137, 4200 ms de inserção e 493, 9290 ms de busca existente, enquanto sua busca inexistente foi de apenas 0, 5330 ms e a remoção de 5, 2405 ms.

A KD-Tree apresentou os maiores tempos de inserção entre as estruturas nesse cenário, com 165, 1880 ms, além de 56, 6389 ms na busca existente e 79, 8414 ms na remoção.

5.9 Discussão dos resultados

Os resultados experimentais mostram que não existe uma única estrutura que apresente o menor tempo em todas as operações e cenários. O comportamento depende do tipo de estrutura, do tamanho da entrada e da distribuição dos dados.

A Trie apresentou comportamento relativamente estável e tempos reduzidos nas quatro operações, especialmente nas maiores entradas. Para $n = 100.000$, no cenário aleatório, seus tempos foram 23, 4639 ms para inserção, 5, 7059 ms para busca existente, 2, 3153 ms para busca inexistente e 22, 8651 ms para remoção.

A Patricia apresentou tempos superiores aos da Trie nas maiores entradas avaliadas. Esse comportamento pode estar relacionado às operações adicionais necessárias para manipular sua representação compactada. No cenário aleatório com $n = 100.000$, por exemplo, a busca inexistente levou 53, 2343 ms e a remoção 97, 3098 ms.

A Splay apresentou forte dependência da ordem dos dados. No cenário aleatório, os tempos de busca existente e remoção para $n = 100.000$ foram 78, 5408 ms e 60, 5319 ms. No cenário ordenado, esses valores foram 7, 1638 ms e 10, 1115 ms. Essa diferença evidencia o efeito da reorganização dinâmica característica da estrutura.

A Treap apresentou os resultados mais sensíveis ao cenário de entrada nas maiores escalas. O caso mais expressivo foi $n = 100.000$, no qual a inserção passou de 147, 2500 ms no cenário aleatório para mais de 1 segundo nos cenários ordenado e invertido. Esse comportamento deve ser registrado como uma característica observada experimentalmente da implementação utilizada, sem ser interpretado isoladamente como uma propriedade universal de todas as Treaps.

A KD-Tree apresentou tempos intermediários em várias operações, mas também sofreu influência da distribuição dos pontos. Como a estrutura trabalha com coordenadas e não somente com uma chave escalar, a forma espacial produzida pelo conjunto de dados influencia diretamente os caminhos percorridos durante as consultas.

De maneira geral, os resultados confirmam que a análise assintótica fornece uma expectativa sobre o crescimento das estruturas, mas não determina sozinha os tempos observados. Constantes de implementação, alocação dinâmica, acesso à memória, rotações, representação dos nós e distribuição das entradas exercem influência significativa.

Também é importante destacar que os valores apresentados correspondem a uma execução do benchmark. Portanto, diferenças muito pequenas, especialmente para $n = 100$, não devem ser interpretadas como evidência definitiva de superioridade entre estruturas. Para uma avaliação estatística mais rigorosa, seria necessário repetir cada experimento várias vezes e utilizar medidas como média, mediana e desvio padrão.

5.10 Limitações experimentais

O benchmark apresenta algumas limitações que devem ser consideradas na interpretação dos resultados. Primeiramente, os tempos correspondem a uma execução, não sendo realizadas múltiplas repetições independentes para calcular intervalos de confiança.

Além disso, não foi realizada uma medição direta do consumo de memória no CSV utilizado. Dessa forma, embora a análise teórica apresente os custos espaciais, não é possível afirmar quantitativamente qual estrutura consumiu mais memória durante a execução.

Outra limitação é que as representações utilizadas não são idênticas entre as estruturas. A Trie utiliza chaves textuais, enquanto Splay e Treap trabalham com chaves inteiras e a KD-Tree utiliza pontos bidimensionais. Portanto, a comparação deve ser interpretada como uma avaliação do comportamento das implementações nas tarefas definidas, e não como uma competição absoluta entre estruturas de naturezas diferentes.

Por fim, o tamanho máximo utilizado foi $n = 100.000$. Um teste adicional com $n = 1.000.000$ foi inicialmente considerado, mas a execução do cenário ordenado da Splay não foi concluída dentro do tempo disponível para o experimento. Por esse motivo, o conjunto final de resultados foi limitado a 100.000 elementos, mantendo-se assim um conjunto completo de medições comparáveis entre as cinco estruturas.

6 Aplicações, Análise Crítica e Discussão

As cinco estruturas estudadas apresentam finalidades diferentes. Consequentemente, uma comparação adequada deve considerar o problema para o qual cada estrutura foi projetada.

6.1 Trie

A Trie é especialmente adequada para aplicações que trabalham com strings e prefixos. Exemplos incluem autocompletar, dicionários, corretores ortográficos, mecanismos de busca por prefixos e algumas formas de indexação textual.

Em um sistema de autocompletar, por exemplo, uma consulta como “comp” pode ser utilizada para localizar rapidamente palavras iniciadas por esse prefixo. A estrutura compartilhada permite que as palavras relacionadas sejam acessadas a partir de um caminho comum.

A principal vantagem da Trie é justamente essa relação direta entre a chave e sua sequência de caracteres. Como desvantagem, a representação pode consumir bastante memória, especialmente quando cada nó mantém uma estrutura fixa de ponteiros para todos os caracteres possíveis.

6.2 Patricia

A Patricia é apropriada para problemas de indexação e recuperação em que a compactação das informações relacionadas a prefixos é importante. Sua origem está associada a problemas de recuperação de informação e indexação [3].

Em comparação com uma Trie convencional, sua principal vantagem está na compactação de caminhos que não apresentam bifurcações. Essa característica pode reduzir a quantidade de nós e, consequente-

mente, o espaço estrutural necessário.

Por outro lado, a implementação é mais complexa. Inserções que produzem divergências parciais exigem operações de divisão e reorganização que não existem de maneira equivalente em uma Trie simples.

6.3 Splay

A Splay é adequada para situações nas quais existe localidade nos acessos, isto é, quando determinados elementos tendem a ser consultados repetidamente ou quando existe correlação entre acessos consecutivos.

A estrutura não necessita de informações explícitas de altura ou fatores de balanceamento. Em contrapartida, o custo de uma operação isolada pode ser alto, e a estrutura pode apresentar formas bastante diferentes após sequências distintas de consultas.

A principal dificuldade de implementação está na correta realização das rotações e dos diferentes casos de *splay*. Uma pequena falha na atualização dos ponteiros pode comprometer a propriedade da árvore.

6.4 Treap

A Treap pode ser utilizada em problemas que necessitam de uma árvore de busca dinâmica com uma estratégia de balanceamento relativamente simples.

Sua principal característica é a combinação das propriedades de BST e heap. A prioridade fornece um mecanismo probabilístico de controle da forma da árvore, enquanto as rotações permitem restaurar as propriedades após inserções e remoções.

A principal limitação está na natureza probabilística do comportamento. A garantia esperada depende das prioridades e não existe uma garantia determinística de altura logarítmica equivalente à de uma AVL.

6.5 KD-Tree

A KD-Tree é adequada para problemas envolvendo pontos e consultas espaciais. Entre as aplicações possíveis estão sistemas de informação geográfica, processamento geométrico, reconhecimento de padrões e busca por vizinhos.

Sua vantagem fundamental é utilizar a própria dimensionalidade dos dados para particionar o espaço. Isso permite evitar a análise de todos os pontos em determinadas consultas.

Entretanto, a eficiência pode diminuir com o aumento da dimensionalidade e com distribuições desfavoráveis dos pontos. Além disso, a implementação de operações como vizinho mais próximo exige cuidados adicionais para decidir quais regiões podem ser descartadas durante a busca.

6.6 Dificuldades encontradas na implementação

As principais dificuldades estão relacionadas às diferenças entre os mecanismos de organização das estruturas.

Na Trie, os desafios concentram-se no gerenciamento dos nós e na remoção correta de caminhos compartilhados. Na Patricia, a necessidade de compactar e dividir prefixos torna as operações de atualização mais complexas.

Na Splay, a maior dificuldade está na implementação correta das sequências de rotações e na manutenção dos ponteiros durante o processo de reorganização.

Na Treap, além da propriedade de BST, é necessário preservar a propriedade de heap associada às prioridades. Isso exige uma integração cuidadosa entre inserção, remoção e rotações.

Na KD-Tree, o desafio adicional está na dimensionalidade dos dados. A estrutura precisa utilizar uma coordenada diferente conforme a profundidade e, nas consultas espaciais, determinar quais regiões precisam continuar sendo exploradas.

6.7 Comparação integrada

As cinco estruturas podem ser organizadas conceitualmente de acordo com o tipo de problema que procuram resolver.

A Trie e a Patricia concentram-se em chaves textuais e prefixos. A Patricia pode ser vista como uma alternativa compactada quando a representação de uma Trie tradicional produz muitos caminhos de grau um.

A Splay e a Treap, por outro lado, são árvores de busca. A diferença fundamental está no mecanismo utilizado para controlar a estrutura. A Splay reorganiza-se em função dos acessos, enquanto a Treap utiliza prioridades e mantém simultaneamente propriedades de BST e heap.

A KD-Tree possui um objetivo distinto: organizar pontos em um espaço multidimensional e permitir consultas espaciais.

Consequentemente, não é adequado afirmar que uma das cinco estruturas é universalmente superior às demais. O desempenho e a adequação dependem do tipo de dado, das operações predominantes, da distribuição das entradas e das propriedades exigidas pela aplicação.

7 Conclusão

O estudo realizado permitiu analisar cinco estratégias diferentes de organização hierárquica de dados: Trie, Patricia, Splay, Treap e KD-Tree. Embora todas possam ser classificadas como estruturas em árvore, seus objetivos e mecanismos internos são bastante distintos.

A Trie utiliza o compartilhamento de prefixos para organizar strings e oferecer consultas relacionadas à composição das chaves. A Patricia amplia essa ideia por meio da compactação dos caminhos, reduzindo estruturas redundantes e introduzindo mecanismos de divisão de prefixos.

A Splay apresenta uma abordagem adaptativa para árvores de busca. Em vez de manter uma condição de balanceamento explícita, utiliza operações de *splay* para reorganizar a estrutura de acordo com os acessos. Essa característica produz custo amortizado logarítmico, embora uma operação isolada possa apresentar custo linear [5].

A Treap combina as propriedades de uma árvore binária de busca com as de um heap. A utilização de prioridades aleatórias produz comportamento esperado logarítmico para as operações fundamentais,

embora o pior caso permaneça linear [4].

A KD-Tree representa uma abordagem ainda diferente, utilizando a dimensionalidade dos dados para particionar o espaço. Sua utilidade está principalmente associada às consultas espaciais e multidimensionais [1].

Os resultados experimentais complementaram a análise teórica ao demonstrar que as cinco estruturas apresentam comportamentos práticos distintos conforme o tamanho e a distribuição dos dados. Foram avaliadas entradas de 100, 1.000, 10.000 e 100.000 elementos, nos cenários aleatório, ordenado e invertido, considerando inserção, busca de elementos existentes, busca de elementos inexistentes e remoção.

A Trie apresentou comportamento relativamente estável entre os diferentes cenários. Para 100.000 elementos no cenário aleatório, por exemplo, seus tempos foram de 23, 4639 ms para inserção, 5, 7059 ms para busca existente, 2, 3153 ms para busca inexistente e 22, 8651 ms para remoção. Esse comportamento é compatível com a característica da estrutura de trabalhar diretamente sobre os caracteres das chaves.

A Patricia apresentou maior custo em diversas operações nas maiores entradas. No cenário aleatório com 100.000 elementos, foram registrados 58, 0239 ms para inserção, 45, 9322 ms para busca existente, 53, 2343 ms para busca inexistente e 97, 3098 ms para remoção. Esses valores ilustram que a compactação da estrutura não implica necessariamente menor tempo de execução em uma implementação concreta.

A Splay apresentou forte influência do padrão de entrada. Para 100.000 elementos, sua inserção foi de 82, 6586 ms no cenário aleatório, 7, 9674 ms no cenário ordenado e 5, 6819 ms no cenário invertido. A busca existente também variou de 78, 5408 ms no cenário aleatório para 7, 1638 ms no cenário ordenado. Esses resultados são coerentes com a natureza autoajustável da estrutura, na qual as próprias operações modificam a forma da árvore.

A Treap apresentou a maior variação entre cenários nas maiores entradas. Para 100.000 elementos, sua inserção foi de 147, 2500 ms no cenário aleatório, mas atingiu 1110, 1600 ms no cenário ordenado e 1137, 4200 ms no cenário invertido. A busca existente também apresentou diferenças substanciais, com 26, 3607 ms no cenário aleatório, 509, 3810 ms no ordenado e 493, 9290 ms no invertido. Esses resultados mostram a influência da forma efetivamente produzida pela implementação durante as operações realizadas.

A KD-Tree apresentou comportamento associado à distribuição dos pontos. No cenário aleatório com 100.000 elementos, foram observados 115, 2340 ms na inserção, 43, 9425 ms na busca existente, 1, 6260 ms na busca inexistente e 47, 2835 ms na remoção. No cenário invertido, a inserção aumentou

para 165, 1880 ms, evidenciando a influência da organização dos dados sobre a estrutura espacial produzida.

Os experimentos também demonstraram que complexidade assintótica e desempenho medido não são conceitos equivalentes. A complexidade fornece uma descrição do crescimento esperado sob determinadas hipóteses, enquanto o tempo observado depende também de constantes, representação dos nós, alocação dinâmica, acesso à memória, distribuição das entradas e sequência de operações.

Como limitação, os resultados apresentados correspondem a uma execução do benchmark e não incluem medição direta de consumo de memória. Dessa forma, os valores devem ser interpretados como uma caracterização experimental das implementações desenvolvidas, e não como uma medição universal das estruturas em qualquer ambiente.

De forma geral, o trabalho evidencia que a escolha de uma estrutura de dados deve estar relacionada às características do problema. A Trie e a Patricia são voltadas a chaves estruturadas por prefixos; Splay e Treap apresentam estratégias distintas para árvores de busca dinâmicas; e a KD-Tree é direcionada a dados multidimensionais. Portanto, não existe uma estrutura universalmente adequada para todos os cenários, sendo necessário considerar o tipo de dado, as operações predominantes e a distribuição das entradas.

8 Referências

Referências

- [1] BENTLEY, Jon Louis. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, v. 18, n. 9, p. 509–517, 1975.
 - [2] FREDKIN, Edward. Trie memory. *Communications of the ACM*, v. 3, n. 9, p. 490–499, 1960.
 - [3] MORRISON, Donald R. PATRICIA – Practical Algorithm To Retrieve Information Coded in Alphanumeric. *Journal of the ACM*, v. 15, n. 4, p. 514–534, 1968.
 - [4] SEIDEL, Raimund; ARAGON, Cecilia R. Randomized search trees. *Algorithmica*, v. 16, p. 464–497, 1996.
 - [5] SLEATOR, Daniel D.; TARJAN, Robert E. Self-adjusting binary search trees. *Journal of the ACM*, v. 32, n. 3, p. 652–686, 1985.
-